

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026– 27

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N Input Size

F Filter Size

P Padding

S Stride

3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,
Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Suppose

Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16$$

$$= 448$$

Therefore,
Total trainable parameters = 448.

7. Dataset

Dataset: CIFAR-10

- Training Images: 50,000
- Testing Images: 10,000
- Classes: 10
- Image Size: $32 \times 32 \times 3$

The ten classes are Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, and Truck.

8. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2

Implement a convolution layer.

Compare

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Task 3

Study hyperparameters.

Compare

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Task 4

Visualize feature maps after the first convolution layer.

Display at least eight feature maps.

Task 5

Compare

- Max Pooling
- Average Pooling

Compare output size and accuracy.

Task 6

Construct the following CNN.

Input $\rightarrow$ *Conv* $\rightarrow$ *ReLU* $\rightarrow$ *MaxPool* $\rightarrow$ *Conv* $\rightarrow$ *ReLU* $\rightarrow$ *MaxPool* $\rightarrow$ *Flatten* $\rightarrow$ *Dense* $\rightarrow$ *Softmax*

Train for

- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

Task 7

Evaluate

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

9. Source Code

The complete implementation of the Single Layer Perceptron, including data preprocessing, exploratory data analysis, model implementation, training, evaluation, visualization, and additional tasks, has been uploaded to GitHub.

GitHub Repository:

<https://github.com/KPR-24110081/Deep-Learning-Lab/tree/main/Ex-3>

The repository contains:

- Jupyter Notebook (`Ex_1.ipynb`)
- Banknote Authentication Dataset
- Generated Plots
- Experiment Report
- README describing the implementation

10. Mandatory Plots

The following plots were generated as part of the experiment. Each plot is followed by its corresponding inference.

10.1 Sample Images



Figure 1: Sample Images from the CIFAR-10 Dataset

Inference: The sample images represent the ten CIFAR-10 classes. Each image has a spatial size of 32×32 pixels and contains three RGB channels.

10.2 Class Distribution

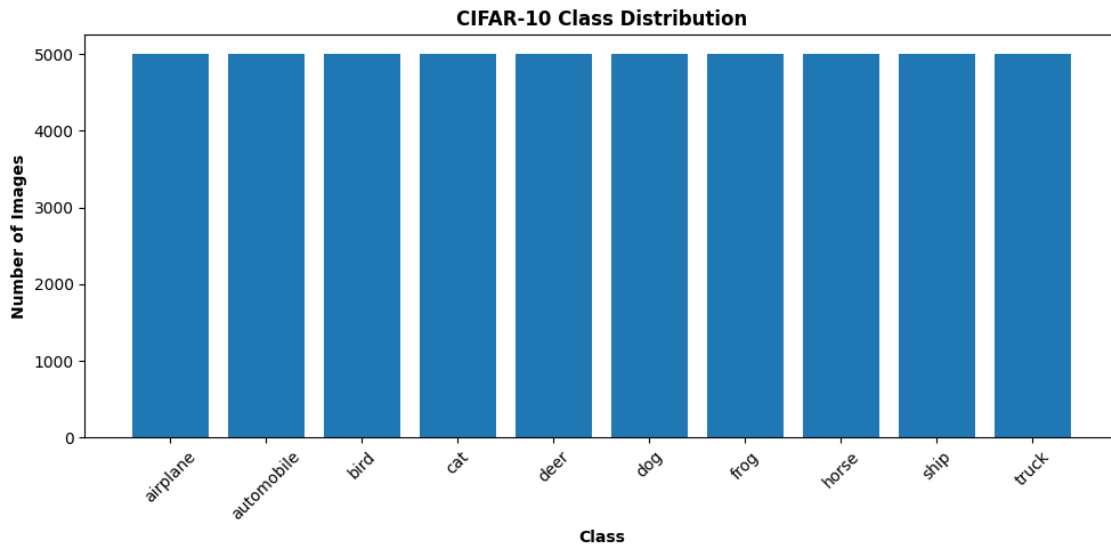


Figure 2: CIFAR-10 Class Distribution

Inference: The class distribution contains approximately 5,000 training images for each of the ten classes. Therefore, the CIFAR-10 training dataset is balanced across all categories.

10.3 Training and Validation Accuracy

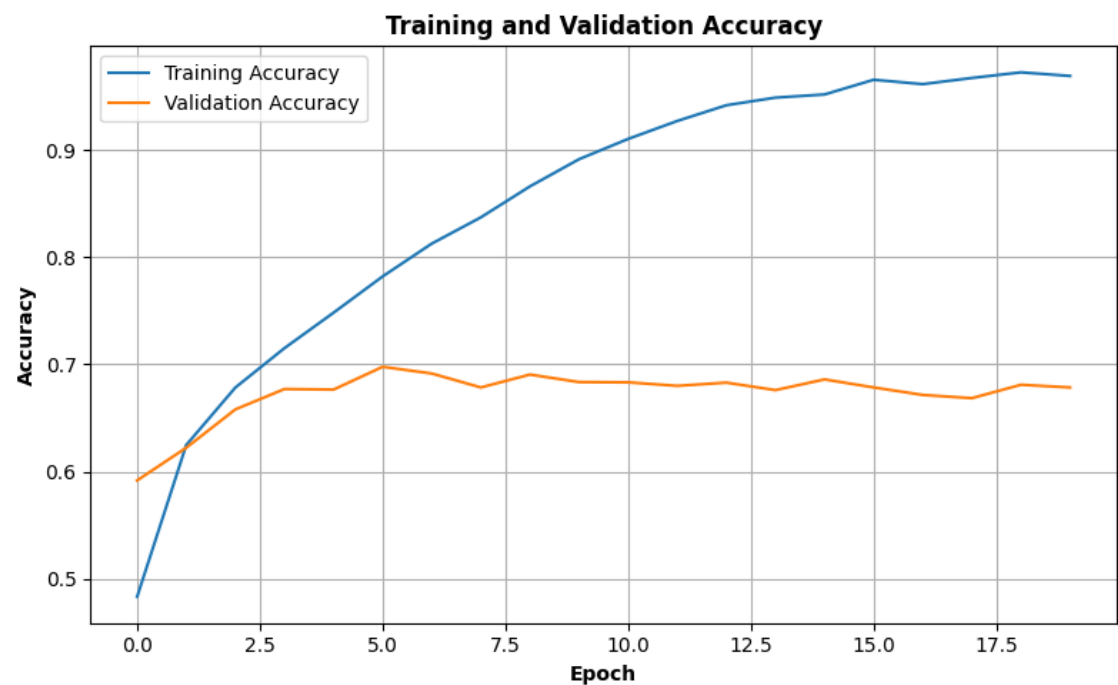


Figure 3: Training and Validation Accuracy

Inference: Training accuracy increased substantially during the 20 epochs and reached approximately 97%. This indicates that the CNN learned the training samples effectively. Validation accuracy improved during the initial epochs but remained considerably below training accuracy. The increasing gap between the two curves indicates overfitting.

10.4 Training and Validation Loss



Figure 4: Training and Validation Loss

Inference: Training loss decreased continuously and reached approximately 0.09 by the final epoch. This shows that the model progressively reduced its error on the training data. Validation loss initially decreased but increased during the later epochs, reaching approximately 2.28 at the final epoch. Its divergence from training loss indicates overfitting.

10.5 Feature Maps

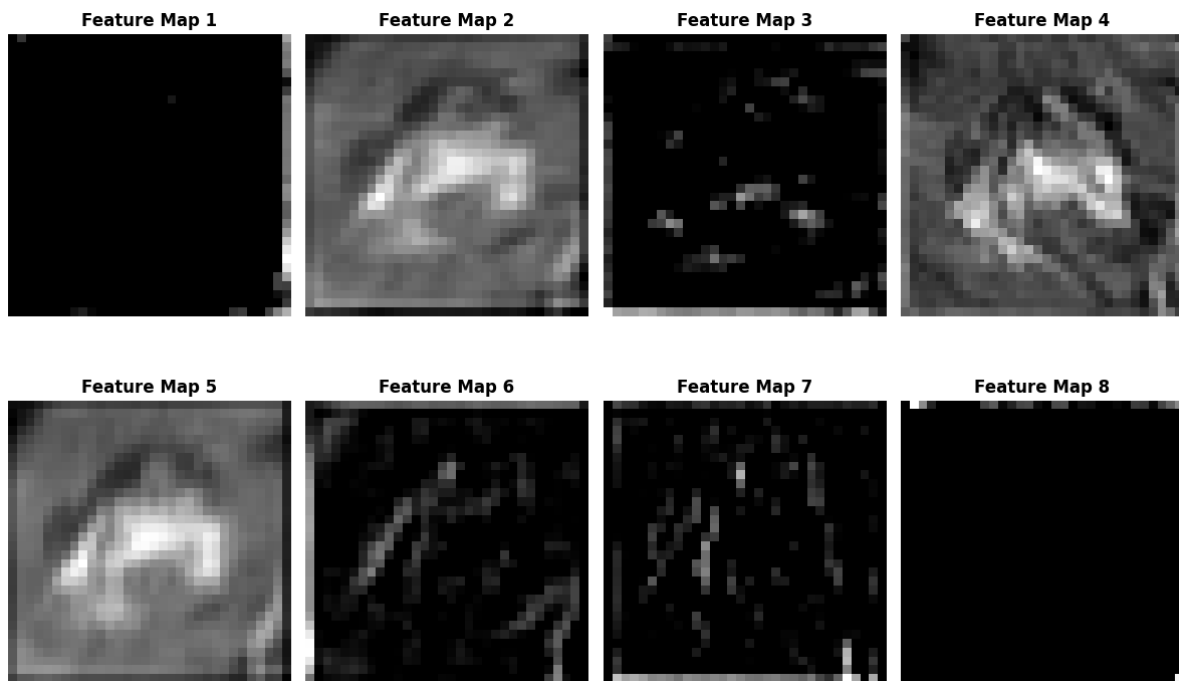


Figure 5: Feature Maps Generated by the Convolution Layer

Inference: Different convolution filters respond to different patterns in the input image. Some feature maps emphasize edges and local structures, while others capture different visual characteristics.

10.6 Confusion Matrix

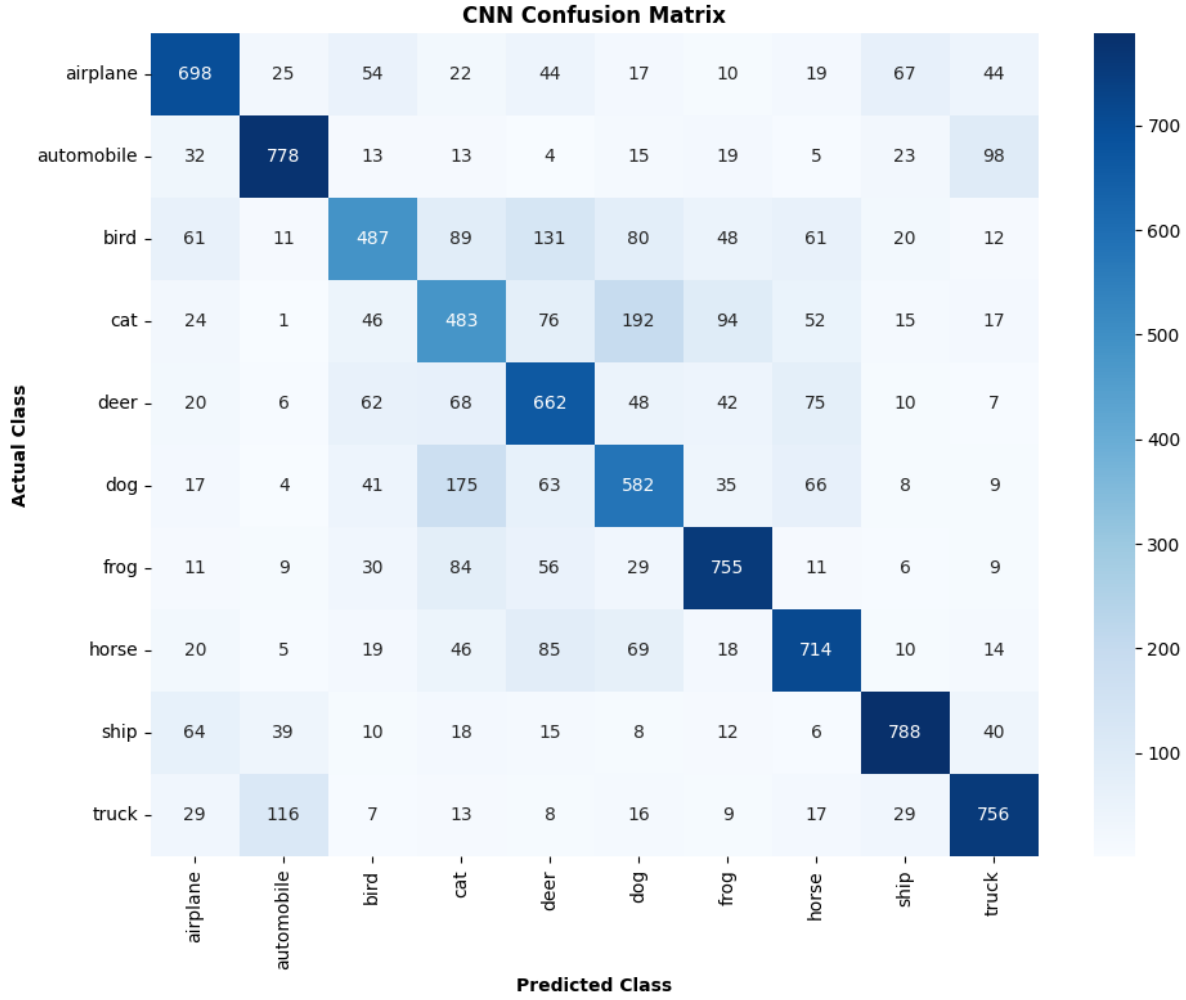


Figure 6: Confusion Matrix of the CNN

Inference: The diagonal elements represent correctly classified images, while the off-diagonal elements represent misclassifications. The matrix shows that classification performance varies across the ten CIFAR-10 classes.

11. Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.

$$O = \left\lfloor \frac{64 - 5 + 2(2)}{2} \right\rfloor + 1 = \left\lfloor \frac{63}{2} \right\rfloor + 1 = 31 + 1 = \boxed{32}$$

Therefore, the output feature-map size is:

$$\boxed{32 \times 32}$$

2. Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.

$$(3 \times 3 \times 3 + 1) \times 64 = (27 + 1) \times 64 = 28 \times 64 = \boxed{1792}$$

Therefore, the layer contains 1,792 trainable parameters.

3. Compare ReLU and Sigmoid activation functions.

$$f_{\text{ReLU}}(x) = \max(0, x)$$

$$f_{\text{Sigmoid}}(x) = \frac{1}{1 + e^{-x}}$$

ReLU produces zero for negative inputs and retains positive values, whereas Sigmoid maps the input to a value between 0 and 1.

Table 1: Comparison of ReLU and Sigmoid

Property	ReLU	Sigmoid
Output Range	$[0, \infty)$	$(0, 1)$
Negative Input	0	Positive value
Typical Use	Hidden Layers	Binary Output
Computation	Simple	More computationally expensive

For the CNN used in this experiment, ReLU was used as the activation function in the convolutional and dense hidden layers.

4. Replace Max Pooling with Average Pooling and compare the results.

Table 2: Pooling Method Comparison

Pooling Method	Test Accuracy
Max Pooling	67.03%
Average Pooling	67.59%

Average Pooling achieved a slightly higher test accuracy than Max Pooling in this experiment. Both methods reduce the spatial dimensions of feature maps, but they summarize activation values differently.

5. Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.

Table 3: Effect of Number of Convolution Filters

Configuration	Test Accuracy	Training Time (s)
16 Filters	63.48%	158.04
64 Filters	67.25%	340.12

Increasing the number of filters from 16 to 64 improved test accuracy from 63.48% to 67.25%. However, training time increased from 158.04 seconds to 340.12 seconds because the larger number of filters increases computational requirements.

12. Results

The performance of the CNN model was evaluated using the training and testing datasets.

Table 4: CNN Performance Results

Metric	Value
Training Accuracy	96.91%
Testing Accuracy	67.03%
Precision	67.18%
Recall	67.03%
F1-score	66.97%
Number of Parameters	545,098

The CNN achieved a training accuracy of 96.91% and a testing accuracy of 67.03%. The precision, recall, and F1-score were 67.18%, 67.03%, and 66.97%, respectively. The constructed CNN contains 545,098 trainable parameters.

13. Discussion

1. Why is convolution preferred over fully connected layers for images?

Convolution is preferred for image data because it exploits the spatial structure of images and focuses on local regions. The same convolutional filter is shared across different locations, which reduces the number of trainable parameters while allowing the network to learn important spatial features.

2. How does stride affect the feature map size?

Stride determines the number of pixels by which the convolution filter moves across the input. Increasing the stride reduces the spatial dimensions of the resulting feature map. Therefore, a larger stride produces a smaller feature map and reduces the number of convolution operations.

3. What is the role of padding?

Padding adds additional pixels around the input before applying convolution. It helps preserve spatial information and control the size of the resulting feature map. Same padding can maintain spatial dimensions under appropriate conditions, whereas valid padding does not add extra pixels.

4. Why is pooling used?

Pooling reduces the spatial dimensions of feature maps while retaining important information. It reduces computational requirements and can make the learned representation less sensitive to small spatial changes in the input.

5. How do feature maps represent image characteristics?

Feature maps represent the responses produced by convolution filters when they are applied to an input image. Different filters learn different visual patterns such as edges, textures, and other local structures. The collection of feature maps therefore provides a representation of characteristics detected by the convolutional layers.

6. Why do CNNs require fewer parameters than MLPs?

CNNs generally require fewer parameters than MLPs because convolutional layers use local connectivity and parameter sharing. A single filter is reused across different spatial locations instead of assigning separate weights to every input pixel and neuron connection.

14. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.